Assignment 5

Name : Ravish Ranjan
Roll no. : 45
Course : MCA
Subject : Data Communications and Computer Networks

Question

Write a Python script to demonstrate a 3-way handshake synchronization process of the TCP protocol between two nodes. Set the Urgent Flag to ‘1’ and Destination Port to '5394’ in the packet. Also, verify the details of the packet received on the receiver end.

Answer

```
from scapy.all import IP, TCP, Raw, send, sniff
import threading
import time

SERVER_IP = "127.0.0.1"
CLIENT_PORT = 12345
SERVER_PORT = 5394

CLIENT_SEQ = 1000
SERVER_SEQ = 2000


def receiver():
    print("\n[Receiver] Waiting for SYN packet...")

    # STEP 1: Receive SYN
    packets = sniff(
        filter=f"tcp and dst port {SERVER_PORT}",
        count=1,
        timeout=15,
        iface="lo"
    )

    if not packets:
        print("[Receiver] SYN packet not received!")
        return

    syn = packets[0]
    tcp = syn[TCP]

    if not tcp.flags.S:
        print("[Receiver] Invalid SYN packet!")
        return
```

```
    print("\n[Receiver] STEP 1: SYN received")
    print("Source Port:", tcp.sport)
    print("Destination Port:", tcp.dport)
    print("Sequence Number:", tcp.seq)

    # STEP 2: Send SYN-ACK
    syn_ack = (
        IP(src=SERVER_IP, dst=SERVER_IP) /
        TCP(
            sport=SERVER_PORT,
            dport=CLIENT_PORT,
            seq=SERVER_SEQ,
            ack=tcp.seq + 1,
            flags="SA"
        )
    )

    print("\n[Receiver] STEP 2: Sending SYN-ACK")
    send(syn_ack, verbose=False)

    # STEP 3: Receive final ACK
    packets = sniff(
        filter=f"tcp and dst port {SERVER_PORT}",
        count=1,
        timeout=15,
        iface="lo"
    )

    if not packets:
        print("[Receiver] Final ACK not received!")
        return

    ack_packet = packets[0]
    ack_tcp = ack_packet[TCP]

    print("\n[Receiver] STEP 3: ACK received")

    print("\n========== RECEIVED PACKET VERIFICATION ==========")
    print("Source IP:", ack_packet[IP].src)
    print("Destination IP:", ack_packet[IP].dst)
    print("Source Port:", ack_tcp.sport)
    print("Destination Port:", ack_tcp.dport)
    print("Sequence Number:", ack_tcp.seq)
    print("Acknowledgment Number:", ack_tcp.ack)

    print("\nTCP FLAGS:")
    print("SYN:", int(ack_tcp.flags.S))
    print("ACK:", int(ack_tcp.flags.A))
    print("URG:", int(ack_tcp.flags.U))

    if ack_tcp.dport == SERVER_PORT:
        print("\nDestination Port 5394 Verified!")

    if ack_tcp.flags.U:
```

```
        print("Urgent Flag Verified!")

    if ack_tcp.ack == SERVER_SEQ + 1:
        print("Acknowledgment Number Verified!")

    print("\nComplete Packet:")
    ack_packet.show()

    print("\n[Receiver] Three-Way Handshake Completed!")


def sender():
    time.sleep(1)
    print("\n[Sender] Initiating TCP Three-Way Handshake")

    # STEP 1: Send SYN
    syn = (
        IP(src=SERVER_IP, dst=SERVER_IP) /
        TCP(
            sport=CLIENT_PORT,
            dport=SERVER_PORT,
            seq=CLIENT_SEQ,
            flags="S"
        )
    )

    print("\n[Sender] STEP 1: Sending SYN")
    send(syn, verbose=False)

    # STEP 2: Receive SYN-ACK
    packets = sniff(
        filter=f"tcp and dst port {CLIENT_PORT}",
        count=1,
        timeout=15,
        iface="lo"
    )

    if not packets:
        print("[Sender] SYN-ACK not received!")
        return

    syn_ack = packets[0]
    tcp = syn_ack[TCP]

    if not (tcp.flags.S and tcp.flags.A):
        print("[Sender] Invalid SYN-ACK!")
        return

    print("\n[Sender] STEP 2: SYN-ACK received")
    print("Sequence Number:", tcp.seq)
    print("Acknowledgment Number:", tcp.ack)

    # STEP 3: Send ACK with URG=1
    ack = (
```

```
        IP(src=SERVER_IP, dst=SERVER_IP) /
        TCP(
            sport=CLIENT_PORT,
            dport=SERVER_PORT,
            seq=CLIENT_SEQ + 1,
            ack=tcp.seq + 1,
            flags="AU",
            urgptr=1
        ) /
        Raw(load=b"TCP Handshake Completed")
    )

    print("\n[Sender] STEP 3: Sending ACK with URG=1")
    send(ack, verbose=False)

    print("\n[Sender] Three-Way Handshake Completed!")


receiver_thread = threading.Thread(target=receiver)
sender_thread = threading.Thread(target=sender)

receiver_thread.start()
sender_thread.start()

sender_thread.join()
receiver_thread.join()
```

Output

```
(sem3) ravish@ravishPC:~/work/sem3 ⎇ (main)> sudo
/home/ravish/.local/share/virtualenvs/sem3-xvFGOLMV/bin/python
./cn/prog/a5.py

[Receiver] Waiting for SYN packet...

[Sender] Initiating TCP Three-Way Handshake

[Sender] STEP 1: Sending SYN

[Receiver] STEP 1: SYN received
Source Port: 12345
Destination Port: 5394
Sequence Number: 1000

[Receiver] STEP 2: Sending SYN-ACK

[Sender] STEP 2: SYN-ACK received
Sequence Number: 2000
Acknowledgment Number: 1001

[Sender] STEP 3: Sending ACK with URG=1
```

```
[Sender] Three-Way Handshake Completed!

[Receiver] STEP 3: ACK received

========== RECEIVED PACKET VERIFICATION ==========
Source IP: 127.0.0.1
Destination IP: 127.0.0.1
Source Port: 12345
Destination Port: 5394
Sequence Number: 1001
Acknowledgment Number: 2001

TCP FLAGS:
SYN: 0
ACK: 1
URG: 1

Destination Port 5394 Verified!
Urgent Flag Verified!
Acknowledgment Number Verified!

Complete Packet:
###[ Ethernet ]###
  dst       = 00:00:00:00:00:00
  src       = 00:00:00:00:00:00
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 63
     id        = 1
     flags     =
     frag      = 0
     ttl       = 64
     proto     = tcp
     chksum    = 0x7cb6
     src       = 127.0.0.1
     dst       = 127.0.0.1
     \options   \
###[ TCP ]###
        sport     = 12345
        dport     = 5394
        seq       = 1001
        ack       = 2001
        dataofs   = 5
        reserved  = 0
        flags     = AU
        window    = 8192
        chksum    = 0xb5cc
        urgptr    = 1
        options   = []
###[ Raw ]###
           load      = b'TCP Handshake Completed'
```

```
[Receiver] Three-Way Handshake Completed!
(sem3) ravish@ravishPC:~/work/sem3 ⎇ (main)>
```